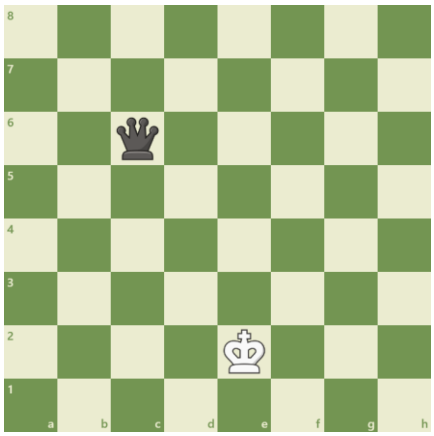
Examen Extraordinario Introducción a la Programación, Ciencia de Datos, 2025-2026

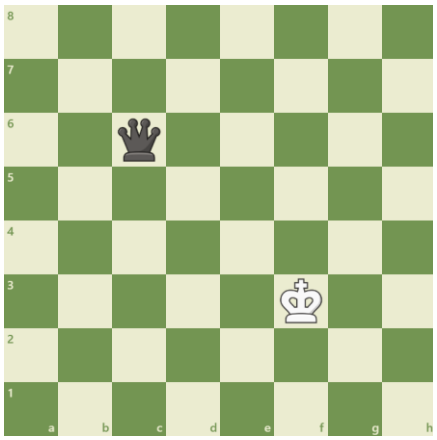
Problema 1: Amenaza de Reina

Descripción

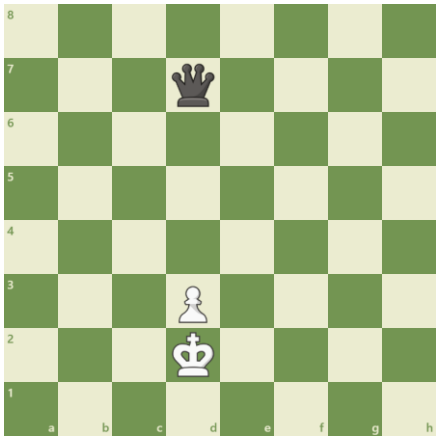
Un aspirante a ajedrecista no conoce bien las reglas del juego. Actualmente está aprendiendo como se mueve la dama y las amenazas sobre otras casillas. Sabe que el tablero de ajedrez tiene 8x8 casillas y según su libro de texto una dama amenaza al rey si está en su columna, fila o diagonal y no existen piezas entre ellos.



NoJaque, no está ni en su horizontal, ni vertical ni diagonal



Jaque, diagonal libre



No Jaque, pieza intermedia

El aprendiz te ha contactado para que construyas un programa que dada la posición en el tablero del rey blanco y de la dama negra devuelva si este está en jaque.

Detalles de Implementación

La función de entrada a la lógica debe tener la siguiente signatura:

```
def dama_amenaza(tablero: List[List[str]]) -> bool:
```

Donde `tablero` es el tablero de juego representado por una lista de listas de string que representan la pieza que está en esa posición. Si la posición está vacía entonces en ese índice existirá el tipo especial `NoneType`.

La codificación de los string que representan una pieza cualquiera es XY, donde X es la letra inicial del nombre de la pieza <u>en mayúscula</u> y Y la inicial del color <u>también en mayúscula</u>. De esta forma el caballo negro se representa como "`CN`", la torre blanca "`TB`", el alfil blanco "`AB`", etc.

Para acceder a una posición cualquiera del tablero se hará de la forma `tablero[i][j]` donde `i` representa el índice de la fila y `j` el índice de la columna. Por lo que el siguiente tablero representa que en la fila 3, columna 4 hay una dama negra porque `tablero[2][3]` es "`DN`".

```
[ [None, None, None, None, None, None, None, None],
 [None, None, None, None, None, None, None, None],
 [None, None, None, "DN", None, None, None, None],
 [None, None, None, None, None, None, None, None],
 [None, None, None, None, None, None, None, None],
 [None, None, None, None. None, "PB", "RB", None],
 [None, None, None, None, None, None, None, None],
[None, None, None, None, None, None, None, None] ]
```

NOTA: Considere que la lista de listas que la función recibe como parámetro representa un tablero correcto, por lo que su código no tiene que verificar ni controlar eso.

NOTA: Para facilitarle sus pruebas puede crear un archivo texto con cada configuración que quiera probar y leerlo mediante el código Python

Problema 2: Tienda y sus ingresos

Descripción

Una tienda tiene una lista de productos sobre los que realiza diariamente compras para abastecer la tienda y ventas para generar ingreso. Los productos se venden a un precio superior al que se compraron para poder generar ganancias. Esta tienda quiere llevar la estadística de sus ingresos y egresos diarios y actualizar su inventario al final del día.

La tienda mantiene un inventario donde para cada producto refleja su disponibilidad, precio de venta y costo real de adquisición. Al final de cada día existe un registro de las operaciones que se hicieron. Cada operación consiste en una acción (compra o venta), un producto y la cantidad que se compró/vendió. La tienda te está contactando para que analices las operaciones del día, actualices su inventario y digas sus ingresos y gastos del día.

Detalles de Implementación

El inventario de la tienda está escrito en un archivo `inventario.json` con el formato

```
{
    "nombre_del_producto_1":
        {
            "disponibilidad": int,# cantidad de unidades del producto
            "precio": float,      # precio de venta al cliente
            "costo_real": float   # cuánto pagó la tienda por el producto
        },
    ...
    "nombre_del_producto_n":
        {
            "disponibilidad": 35,
            "precio": 100,
            "costo_real": 80
        }
}
```

Las operaciones de la tienda del final del día están escritas en un archivo `operaciones.json` con el formato

```
[
    {   "acción": str,      # compra o venta
        "producto": str,  # nombre del producto
        "cantidad": int   # cantidad del producto comprada/vendida
    }, {
        "acción": "compra",
        "producto": "Manzanas",
        "cantidad": 200
    }
    ...
]
```

La función de entrada a la lógica debe tener la siguiente signatura:

```
def actualiza_inventario() -> Tuple[float, float]:
```

Donde se devuelve una tupla cuyo primer valor es la ganancia que se tuvo de todos artículos que se vendieron en el día; y el segundo valor representa la cantidad de dinero que fue invertido en comprar nuevos productos. Recuerde que la ganancia se determina como
`precio - costo_real`

Su función debe modificar internamente el `inventario.json` para actualizar la disponibilidad de los productos

Restricciones

`precio > costo_real`

No se venderán más productos que la disponibilidad que exista en el inventario. Las operaciones de venta registradas son correctas.

NOTA: No existen límites para las operaciones de compra de productos para abastecer el inventario.

Problema 3: Joyero cortando cadena

Descripción

Un joyero posee una cadena de oro en la que en cada eslabón está incrustada una gema. Cada gema tiene un valor de beneficio (positivo si es valiosa, negativo si está dañada o reduce el atractivo). El joyero desea cortar un segmento continuo de la cadena para venderlo a su cliente y obtener el máximo beneficio posible.

Sin embargo, el cliente puede imponer restricciones:

- Puede no querer gemas más caras que un valor max_valor_gema.
- Puede no querer gemas más baratas que un valor min_valor_gema.

El joyero te está contactando para que lo ayudes porque solo confía en un algoritmo recursivo que use la estrategia de divide y vencerás y sabe que tú eres el único capaz de ayudarlo.

Detalles de implementación

La función de entrada a la lógica debe tener la siguiente signatura:

```
def joyero(
        cadena: List[float],
        max_valor_gema: float = float("inf"),
        min_valor_gema: float = float("-inf")
        ) -> Tuple[float, int, int]:
```

Donde `cadena` representa de forma ordenada el valor de las gemas que están incrustadas en cada eslabón. Y los parámetros `max_valor_gema` y `min_valor_gema` reflejan que no puede existir una gema en la cadena con valor mayor a menor a estos.

La función devuelve una `Tuple[float, int, int]` que se interpreta como (X, Y, Z) equivalente a *"La subcadena de valor X que empieza en el eslabón con índice Y y tiene Z eslabones"*. De no encontrarse ninguna cadena que cumpla la restricción, el índice debe ser -1 y la cantidad de eslabones debe ser 0

Ejemplos

```
resp1 = joyero([5,-1,3,-6,3]) # (7, 0, 3)
resp2 = joyero([-3,-1,-2,-6,-4], min_valor_gema=0) # (-inf, -1, 0)
resp3 = joyero([-3,-1,-2,-6,-4], -2, -4) # (-2, 2, 1)
resp4 = joyero([3,1,2,6,4], 4, 2) # (4, 4, 1)
resp5 = joyero([3,1,2,6,4], max_valor_gema=4, min_valor_gema= 1) # (6, 0, 3)
resp6 = joyero([3]) # (3, 0, 1)
resp7 = joyero([5], 4) # (-inf, -1, 0)
resp8 = joyero([-1, -6, -3, -5, -2, -7, 8, 1, 3]) # (12, 6, 3)
```

Restricciones

```
min_valor_gema <= max_valor_gema
```